Powerset

AlgoExpert · Medium · Recursion

The problem

- Every subset of a set of distinct ints (iteratively doubling).

Intuition

- Each new element doubles the subsets: every existing subset with or without it.

Brute force vs optimal

- Bitmask: each of 2^n masks picks a subset
- Iterative doubling or recursion
- $O(n \cdot 2^n)$ time and space either way

Approach

- 1. Start with the empty set
- 2. For each element, copy all subsets and add it
- 3. Return all

Worked example

```
subsets: [ [] ]
```

Start with the empty set

Worked example

```
add 1 to each existing subset:

[[], [1]]
```

After 1: 2 subsets

Worked example

```
add 2 to each existing subset:

[[], [1], [2], [1, 2]]
```

After 2: 4 subsets

Worked example

```
add 3 to each existing subset:

[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
```

After 3: 8 subsets

Java solution

```java
public class Powerset {
    public static List<List<Integer>> powerset(List<Integer> array) {
        List<List<Integer>> subsets = new ArrayList<>();
        subsets.add(new ArrayList<>());
        for (int x : array) {
            int size = subsets.size();
            for (int i = 0; i < size; i++) { List<Integer> s = new ArrayList<>(subsets.get(i)); s.add(x); subsets.add(s); }
        }
        return subsets;
    }
}
```

Complexity

- Time: O(n * 2^n)
- Space: O(n * 2^n)

Edge cases & pitfalls

- Iterate over the size *before* adding
- Include the empty set
- Duplicates in input → dedupe

Interview tips & follow-ups

- Mention bitmask enumeration
- Follow-up: subsets with duplicates, subsets summing to k

Recap

- Each new element doubles the subsets: every existing subset with or without it.
- Complexity: O(n * 2^n)